

Smart HAS

Oracle PL/SQL integrado ao back-end Java: alertas, relatórios e indicadores direto no banco

GitHub: <https://github.com/GabrielGaruti/smarthas>

Vídeo: [inserir link do vídeo]



Gabriel Garuti Paiva Cracco

RM 554866

Grupo de 1 integrante

O que a Fase 6 entrega

7

tabelas Oracle

modelo físico com
constraints, índices e log de
auditoria

8

rotinas PL/SQL

4 functions e 4 procedures
com cursor, loop, IF e
exceptions

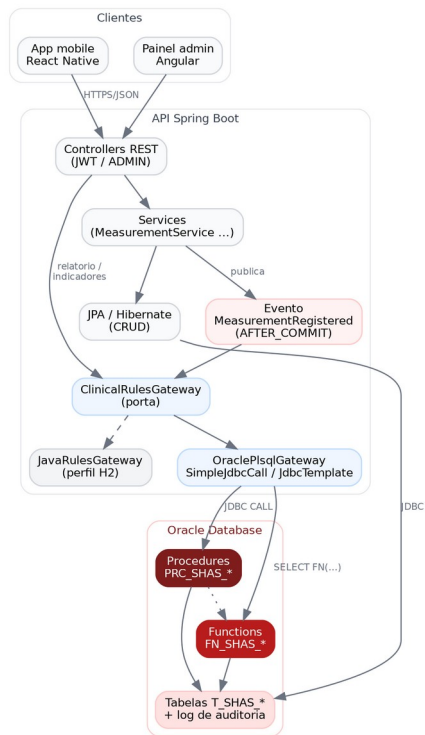
1

fluxo integrado

REST → Java → JDBC →
Oracle, disparado por evento
de back-end

Mobile (React Native) e painel (Angular) continuam consumindo a mesma API Spring Boot.

Uma porta, dois motores de regras



ClinicalRulesGateway

A aplicação só conhece esta interface. O Spring injeta o adaptador conforme o perfil.



Perfil oracle

OraclePlsqlGateway chama procedures com SimpleJdbcCall e functions via SELECT.



Perfil padrão (H2)

JavaRulesGateway replica as mesmas regras para desenvolver sem o servidor Oracle.

Novas funcionalidades e refatorações



Alertas automáticos

Toda leitura é avaliada; o alerta volta na resposta do POST



Relatório consolidado

Gerado no painel admin por procedure Oracle



% de leituras na meta

Indicador de 30 dias no app e no dashboard



Sensores IoT

Dispositivos, origem da leitura e frequência cardíaca



Evento de domínio

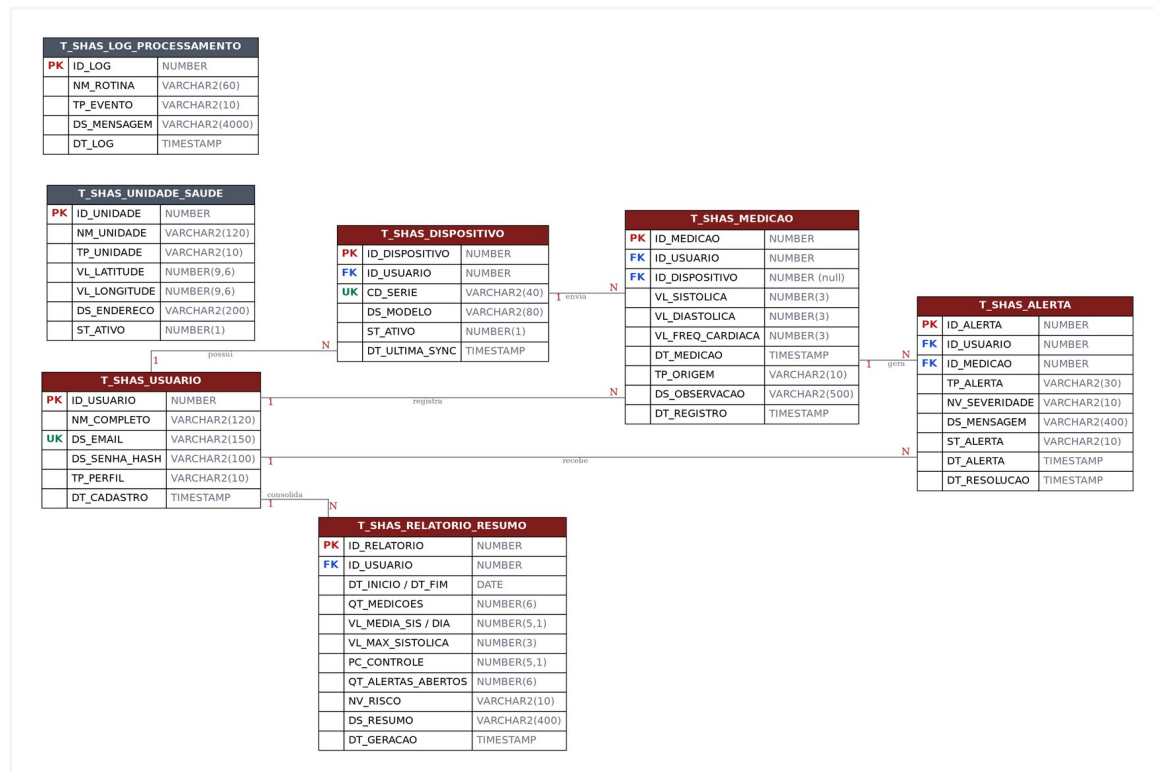
AFTER_COMMIT +
REQUIRES_NEW: regra não desfaz a medição



Robustez

LocalDateTime, erros ORA-20xxx
→ HTTP 400, credenciais por variável

Modelo relacional do Smart HAS



Carga simulada

6 usuários

4 dispositivos IoT

37 leituras simuladas

5 unidades de saúde

Datas relativas a SYSDATE: os indicadores de 30 dias sempre têm dados.

Indicador e dados formatados



FN_SHAS_TAXA_CONTROLE

Indicador: % de leituras abaixo de 140x90 em N dias.
Retorna NULL sem dados; erros -20001 e -20002.



FN_SHAS_RESUMO_PACIENTE

Dados formatados: texto pronto para o app,
reaproveitando as outras functions.

+ FN_SHAS_CLASSIFICAR e FN_SHAS_FORMATAR_MEDICAO
(apoio)

```
-- functions dentro do SELECT
SELECT u.NM_COMPLETO,
       FN_SHAS_TAXA_CONTROLE(
         u.ID_USUARIO, 30) AS NA_META,
       FN_SHAS_RESUMO_PACIENTE(
         u.ID_USUARIO, 30) AS RESUMO
FROM T_SHAS_USUARIO u
WHERE u.TP_PERFIL = 'USER';
```

```
-- mesma consulta usada em
-- GET /admin/patients/indicators
```

PRC_SHAS_REGISTRAR_ALERTA

CRÍTICO

≥ 180x120

Crise hipertensiva

ALTO

≥ 140x90 e 3 das 5 últimas

Tendência alta (CURSOR + LOOP)

MODERADO

≥ 140x90

Hipertensão isolada

*Idempotente (DUP_VAL_ON_INDEX), sem COMMIT interno,
NO_DATA_FOUND → -20010, log autônomo em erro.*

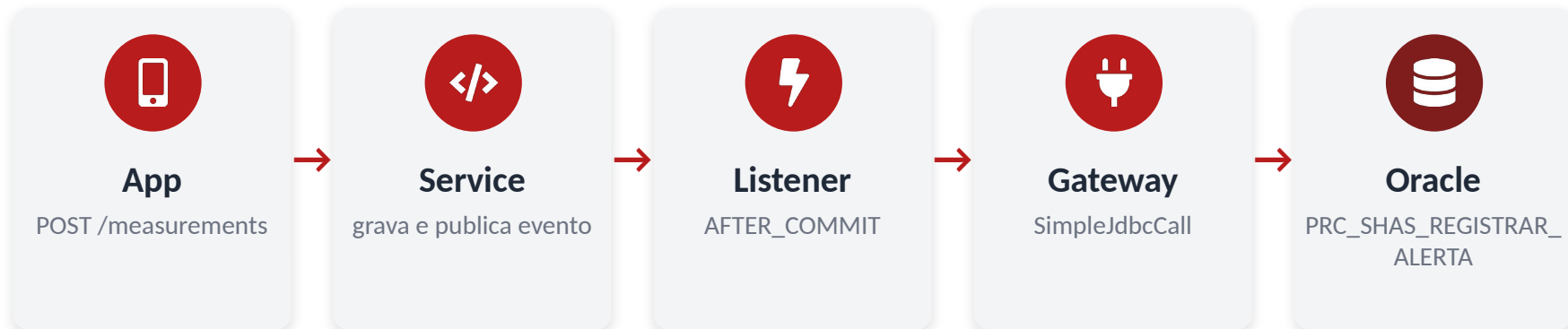
PRC_SHAS_GERAR_RELATORIO_RESUMO

Cursor FOR LOOP por paciente, usa as functions, classifica o risco e trata erro por paciente sem parar o lote.

PRC_SHAS_SIMULAR_LEITURAS_SENSOR

Importa leituras IoT com DBMS_RANDOM e reaproveita a regra de alerta em cada uma.

REST → Java → JDBC → Oracle



O alerta volta no JSON da própria requisição e o app avisa o paciente na hora.



No painel, `POST /admin/reports/summary` executa a procedure de relatório (perfil ADMIN).

O sistema rodando

1 SQL Developer



Script 05: functions no SELECT, procedures e cenários de erro tratados

2 App mobile



Registrar 186x121 e receber o alerta CRÍTICO na hora

3 Painel Angular



Gerar o relatório e ver o motor ORACLE_PLSQL e o tempo de execução

4 Conferência



Novos registros em T_SHAS_ALERTA e T_SHAS_RELATORIO_RESUMO

Inteligência perto dos dados

Próximos passos

- Agendar o relatório com DBMS_SCHEDULER
- Agrupar as rotinas em um PACKAGE
- Receber leituras reais por Bluetooth
- Push quando houver alerta crítico

Gabriel Garuti Paiva Cracco

RM 554866

github.com/GabrielGaruti/smarthas

Vídeo: [\[inserir link do vídeo\]](#)